Práctica 1A

TypeScript

Entrega

- ⇒ Para la entrega el código se realizará por git, indicando el enlace en un documento de texto en moodle.
- ⇒ No se permiten entregas fuera de plazo.
- ⇒ Aquellas prácticas sospechosas de haber sido copiadas, no serán evaluadas.

Ejercicio 1

Crear un nuevo fichero de tipo TS llamado **ejercicio1.ts**, el cuál será transpilado a JS, teniendo en cuenta los siguientes puntos:

1. Se define un tipo llamado Ranking, que es una tupla que contiene tres elementos: un número, una cadena de texto y un booleano. Esta tupla representará un registro en el ranking de un juego.
2. Se declaran varias variables:
   - position: como un número, que almacenará la posición del jugador en el ranking.
   - playerName: como una cadena de texto, que contendrá el nombre del jugador.
   - finishedGame: como un booleano, indicando si el jugador ha finalizado el juego o no.
   - ranking: como una variable del tipo Ranking, que almacenará temporalmente la información de posición, nombre y estado del juego del jugador.
   - hallOfRanking: como un array de Ranking inicializado como vacío, que representará el conjunto de rankings de los jugadores.
3. Se crea un ranking para dos jugadores específicos:
   - Se asignan valores a position, playerName y finishedGame.

- Se crea la tupla ranking con estos valores.
- Se añade esta tupla al array hallOfRanking utilizando el método push().

4. Se define una función llamada printRankings que recibe un array de Ranking como parámetro. Esta función no devuelve ningún valor (void). Su objetivo es imprimir en la consola cada ranking del array proporcionado.
5. Se invoca la función printRankings pasando como argumento el arreglo hallOfRanking, lo que dará como resultado en la consola de los rankings de los jugadores almacenados en dicho array.

Ejercicio 2

Crea un nuevo fichero llamado **ejercicio2.ts** que tendrá dos funciones:

1. Se necesita una función llamada contarPalabras que reciba una cadena de texto como parámetro y cuente la cantidad de palabras que contiene. Se considera que una palabra está delimitada por espacios en blanco.
   La función deberá devolver un número que represente la cantidad de palabras presentes en la cadena de texto proporcionada. Invócala con diferentes textos.

2. Crea una función llamada getAvenger que reciba tres parámetros:
   - Nombre: obligatorio.
   - Poder opcional.
   - Arma: por defecto “arco”.

Esta función devuelve un mensaje que describe las habilidades y armamento del superhéroe basado en los parámetros proporcionados. Si se proporciona un poder, se muestra junto con el nombre y el arma. En caso contrario, se indica únicamente el nombre.
Invoca la función varias veces para comprobar que funciona.

Ejercicio 3

Crea un nuevo archivo TS, llamado **empleado.ts**. En este se definirá una clase Employee con las siguientes características:

- Tendrá varios atributos: nombre, apellido, edad, fecha de inicio del trabajo, una fecha de finalización (podrá ser nula) y la profesión.
- Crea su constructor.
- Crea los métodos get y set para cada uno de los atributos.
- Crea dos empleados, uno que tenga fecha de finalización y otro que no la tenga ya que continúa trabajando, pinta por consola el nombre y la edad de cada uno de ellos.

Ejercicio 4

Se requiere modelar una jerarquía de clases que representen diferentes animales. Para ello, se debe crear una clase padre llamada Animal que incluya atributos y métodos generales que serán heredados por las clases hijas. La clase Animal debe tener como mínimo los siguientes atributos y métodos:

- Atributos:
  - nombre: Tipo string para representar el nombre del animal.
  - edad: Tipo number para representar la edad del animal.
  - tipo: Tipo string para indicar el tipo o especie del animal.
- Métodos: constructor, los getter y setter y método hacerRuido.

Además, se deben crear dos clases hijas de Animal, por ejemplo Perro y Pájaro, que hereden de la clase Animal y agreguen atributos y métodos específicos para cada uno de ellos. Cada clase hija debe tener al menos un método adicional y un atributo propio.

Ejercicio 5

Se necesita modelar una interfaz para definir la estructura de objetos que representan dispositivos electrónicos. Se solicita crear una interfaz llamada DispositivoElectronico que describa los atributos y métodos comunes a todos los dispositivos electrónicos. La interfaz debe incluir al menos los siguientes elementos:

- Atributos:
  - nombre: Tipo string para representar el nombre del dispositivo.
  - encendido: Tipo boolean para indicar si el dispositivo está encendido o apagado.
- Métodos:
  - encender: Método para encender el dispositivo.
  - apagar: Método para apagar el dispositivo.

Además, se debe implementar la interfaz DispositivoElectronico en al menos dos clases, por ejemplo Laptop y Smartphone, que representen dispositivos específicos. Estas clases deben heredar de la interfaz DispositivoElectronico y proporcionar implementaciones concretas para los métodos encender y apagar, junto con otros atributos y métodos propios de cada dispositivo.

- Laptop:
  - Atributo: modelo.
  - Método: abrirPrograma.
- Smartphone:
  - Atributo: marca.
  - Método: llamar.

Debes crear instancias de las clases implementadas y mostrar ejemplos de uso de los métodos encender y apagar, así como de los métodos adicionales definidos en las clases.

Ejercicio 6

Se requiere implementar una clase genérica llamada Almacen que funcione como un contenedor para almacenar diferentes tipos de elementos. Esta clase debe ser capaz de almacenar y recuperar elementos de cualquier tipo de manera segura.

La clase Almacen debe tener una estructura genérica para manejar diferentes tipos de elementos. Debe incluir al menos los siguientes métodos:

- agregarElemento: Método para agregar un elemento al almacén.
- obtenerElemento: Método para obtener un elemento del almacén por su posición.
- listarElementos: Método para listar todos los elementos almacenados en el almacén.

Debes crear instancias de la clase Almacen con diferentes tipos de elementos (números, cadenas, objetos, etc.) y mostrar ejemplos de uso de los métodos implementados, agregando elementos, recuperándolos y listándolos.